High-throughput inference under provider limits

Background

External inference providers make AI models available under capacity limits. These limits are normally defined as **requests per minute (RPM)** and **tokens per minute (TPM)**. Different models can have different limits, and those limits can change over time.

Assume that our available provider capacity is currently:

- **50,000 requests per minute**
- **100 million tokens per minute**

The provider has enough capacity for this traffic, but our current request path does not use it effectively. Requests are sent synchronously, one at a time. A simplified version looks like this:

PY THON / CURRENT REQUEST LOOP

```python
import time
import requests

URL = "http://localhost:8000/inference"

for request_number in range(10_000):
    payload = {
        "request_id": f"req-{request_number}",
        "model": "model-a",
        "estimated_tokens": 1_000,
    }
    requests.post(URL, json=payload).raise_for_status()
    time.sleep(0.1)
```

Ignoring response time, this loop can send roughly **600 requests per minute**. In practice, it will usually be slower. Most of the available provider capacity is therefore left unused.

The problem is to build a system that can process inference requests at much higher rates while respecting the current capacity of each model.

No internal knowledge or specific technology is required.

© 2026 Consuma Technologies Pvt. Ltd.

What you're building

Build a working inference-processing system and a provider simulation. All design choices are yours.

The system should:

- accept individual and batch requests with a model, payload, and estimated token count
- respect changing per-model RPM and TPM limits, expose status and results, and define behavior when demand exceeds capacity

Scale goals

- **Provider-capacity test:** use the initial 50,000 RPM and 100M TPM limits and show that the system can use most of this capacity without exceeding it.
- **Required simulation benchmark:** complete 300,000 simulated inference requests per second.
- **Stretch simulation benchmark:** complete 1,000,000 simulated inference requests per second.
- **Planning target:** explain how the design would be extended toward 10 billion inference requests per minute.

For the high-throughput benchmarks, you may give the simulated models higher limits. You are not expected to make real, paid LLM calls at these rates.

A request counts as completed only when the simulated provider call reaches a final success or failure state. Merely accepting the request at an API does not count as processing it.

Asynchronous batch behavior

The system should support asynchronous batch submission.

A client should be able to submit a batch with a ***callback_url***. The service should quickly return an acknowledgement containing a batch id, process the requests in the background, and call the supplied URL after every request in the batch has reached a final state.

The callback should identify the batch and include:

- the overall batch status
- total, successful, and failed request counts
- either the results or a way to retrieve them

If callback delivery fails, the system should make a reasonable effort to deliver it again. The client should also be able to query the batch status independently.

Provider simulator and load generator

Provide a simulator with configurable models, RPM and TPM limits, response latency, and response outcomes. Also provide a load generator that lets the reviewer choose the request rate, duration, model mix, batch size, and approximate token size.

© 2026 Consuma Technologies Pvt. Ltd.

At the end of a run, report submitted, completed, successful, and failed requests; achieved completion rate; p50, p95, and p99 latency; configured and observed RPM and TPM per model; and batch completion and callback timing.

Validation scenarios

Your submission should include repeatable evidence for the following scenarios.

Scenario 1: Reach the available provider capacity

Configure a model with a limit of 50,000 RPM and 100M TPM. Submit one thousand-token requests for at least five minutes at a rate higher than the configured capacity.

The scenario passes when:

- the system completes at least 90% of the allowed request capacity after warm-up
- no observed 60-second period exceeds either configured limit
- every submitted request is accounted for as completed, waiting, expired, or rejected

Scenario 2: Different models and changing limits

Run traffic against:

- Model A: 30,000 RPM and 60M TPM
- Model B: 20,000 RPM and 40M TPM

Reduce Model A to 5,000 RPM during the test, then restore it. Leave Model B unchanged.

The scenario passes when:

- each model remains within its current limits
- Model A adjusts to both changes without restarting the system
- Model B continues processing while Model A is constrained
- the report shows configured limits and observed throughput over time

Scenario 3: Asynchronous batch completion

Submit 10,000 requests across two models with a callback URL and simulated transient and permanent failures. Reject the first two callback delivery attempts.

The scenario passes when:

- the initial acknowledgement is returned within one second
- the callback is sent only after all requests reach a final state
- callback delivery succeeds after the destination recovers
- the callback summary matches the batch status returned by the API
- every request id appears exactly once in the final batch result

© 2026 Consuma Technologies Pvt. Ltd.

Please ensure your solution...

- is easy for another engineer to run and review, respects configured limits, and tracks every request through failures
  reports measurements separately from projections and states the simulation •
  assumptions

You are free and encouraged to use Claude Code, Cursor, Codex, or any other AI tools. AI makes it possible to attempt substantially more than the bare minimum, so use it to raise the ambition, completeness, and polish of your submission. You remain responsible for understanding the result and explaining the decisions you made.

Deliverables

- A working implementation in a GitHub repository.
- A ***README.md*** describing:
  - how to run the service, provider simulator, and load generator
  - how to configure models and change their limits
  - how to run each validation scenario
  -

key design decisions and tradeoffs

- A benchmark report covering the test environment, configuration, achieved throughput, latency, failures, and bottlenecks.
- The scripts or configuration required to reproduce the results.
- A **3-4 minute Loom video**:
  - show the system processing requests under load
  - change a model's limits while the system is running
  - show an asynchronous batch and its completion callback
  - explain the measured results and your main decisions
  - briefly show how you used AI tools and what you personally contributed